

Padrões de desenvolvimento

Aqui estão registrados os padrões de branch e commit utilizados pela EJCM. É essencial que esse padrão seja seguido durante todo o projeto.

Clean code

Idioma:

- Sempre utilizar o mesmo idioma para comentários, nomeação de funções, arquivos, variáveis e, até mesmo, para tudo que envolve o git (nome de branch, mensagem de commit e etc...).
- Para o nosso projeto, o idioma padrão definido é o português para documentação (figma, README, modelagem BD, comentários no código) e o inglês para a parte operacional (mensagens de commit, código ou seja: nome de variáveis, funções, ...).

Regras de nomeação

- Classes e objetos
 - Classes e objetos devem ter nomes com substantivos
 - Não deve ser um verbo
 - Começam com letra maiúscula
 - Camelcase
- Métodos
 - Devem ser verbos
 - Começam com letra minúscula
 - Camelcase
- Variáveis
 - Começam com letra minúscula
 - Camelcase

Boas práticas

- Usar nomes que revelam seu propósito
 - O nome de uma variável, função ou classe deve responder todas as grandes questões sobre elas
 - Deve dizer porque existe, o que faz e como é usado
 - Se um nome requer um comentario, então ele não revela seu propósito e deve ser revisado
 - Facilita alterações e entendimento
 - Não é sobre simplicidade do código, e sim sobre estar explícito o que ele faz
- Fazer distinções significativas
 - Como não é possível usar o mesmo nome para referir-se a duas coisas diferentes num mesmo escopo, você pode decidir alterar o nome de maneira arbitrária
 - Não usar números sequenciais na nomeação de variáveis
- Evitar o mapeamento mental
 - Os leitores não devem ter que traduzir mentalmente os nomes que você escolheu por outros que eles conheçam
 - Esse é um problema com nomes de variáveis de uma só letra

- É aceito que um contador seja chamado de "i", "j" ou "k", mas somente se o seu escopo for muito pequeno e não houver outros nomes que possam entrar em conflito com ele
- Na maioria dos contextos, um nome de uma letra só é uma escolha ruim
- Selecionar uma palavra por conceito
 - Escolha uma palavra por cada conceito abstrato e fique com ela
- Evitar desinformação
 - Devemos evitar passar dicas falsas que confundam o sentido do código
 - Evitar usar palavras com duplo significado que pode desviar o entendimento
 - Não use list no nome das variáveis, a não ser que realmente seja uma lista, pois a palavra list significa algo específico para programadores
 - Cuidado com nomes muito parecidos
 - Cuidado com as letras "l" e "O"
- Coloquialismos, gírias e trocadilhos no código
 - Não use
- Usar nomes pronunciáveis
 - Isso importa porque a programação é uma atividade social
 - Cuidado com abreviações
 - Antes um nome grande, do que impronunciável

Funções

- Pequenas!
 - A primeira regra para funções é que elas devem ser pequenas
 - Quão pequenas? Vai do bom senso
- Fazer apenas uma coisa
 - Funções devem fazer uma única coisa (e fazê-la bem)
 - O que significa exatamente "uma coisa"? Podem ser uma sequência de ações com a mesma finalidade, dentro do mesmo contexto
- Blocos e indentação
 - O ideal é que o nível de indentação das funções seja um ou dois, pois facilita a leitura
 - Blocos if, else e while não devem ser muito grandes
 - Evitar estruturas aninhadas
- Seções dentro de funções
 - Declarações, inicializações e execução
- Parâmetros
 - Cuidado!
 - Ideal nulo, no máximo 3
 - Ter um motivo muito especial para passar de 3 parâmetros
 - Parâmetros lógicos
 - Evitar usar parâmetros booleanos
 - Complica automaticamente a assinatura do método
 - A função faz mais de uma coisa (se true faz uma coisa, se false faz outra)
- Evite efeitos colaterais!
 - Cuidado com variáveis de classe!
 - Sincronismo
- Separação comando & consulta
 - As funções devem fazer ou responder algo, nunca ambos

- Extraia os blocos try/catch
 - Esses blocos confundem a estrutura do código e misturam o tratamento de erro com o processamento normal do código
 - Portanto, é melhor colocar as estruturas try e catch em suas próprias funções

Formatação:

- Utilizar um padrão de nomeação: como o padrão da EJC é **camelCase**, usem tanto no back quanto no front, apenas quando se tratar de classes e interfaces, utilizem **PascalCase**.
- Classes e objetos: Utilizar **substantivos**.
- Funções: Utilizar **verbos**.
- Escolher nomes que sejam descritivos, que expliquem de forma detalhada o que está ocorrendo.
- Evitar escrever piadas para nomeação.
- Lembrar sempre de formatar os arquivos com o **Ctrl+Shift+I** no VSCode.

Back-End

A pasta de back-end já está com a organização básica estruturada e está usando o **npm** como package manager, então os comando são:

npm install: para adicionar algum package ou atualizar os da máquina.

npm run migrate: para criar o banco de dados quando os códigos estiverem prontos.

npm run seed: para popular o banco de dados.

npm run keys: para quando a autenticação estiver pronta.

npm run dev: para inicializar o banco na máquina de vocês.

Front-End

Na pasta de front tem a estrutura básica do aplicativo e o package manager utilizado é o também é o npm, então os comandos são:

npm install: adicionar algum pacote

npm run dev: inicializar a aplicação web

alternativamente podemos usar yarn install / yarn start

Padrões do Git

/icons/warning_blue.svg ****NOSSOS COMMITS E BRANCHES SERÃO ESCRITOS EM INGLÊS.****

Com relação a organização do Git, acontece da seguinte forma:

Quando vocês quiserem criar uma branch nova, deve-se entrar na branch **develop** com o comando git checkout develop e, a partir dela, criar a branch com o comando git checkout -b develop nome-da-branch-nova utilizando essa sintaxe que pegará tudo que estiver dentro da develop e copiar para a branch nova.

Ao criar as alterações, deverão utilizar o comando git add . ou git add nome-do-arquivo para adicionar o(s) arquivo(s) para commit, e então deve-se adicionar a mensagem de commit com o comando git commit -m

"texto do commit".

Por fim, basta atualizar o repositório remoto com as suas alterações locais com o comando `git push origin nome-da-branch-atual`.

Pode dar quantos pushes desejar e commitar a vontade. Lembre-se que quanto mais commits tivermos, menos retrabalho terá caso dê algum problema no projeto.

Branches

Para a parte do Git, lembrem-se de utilizar camelCase tanto nos commits quanto nas branches.

feature/escopo/autor/descrição

git checkout -b tipo/escopo/nome/descrição - para criar uma branch e alternar pra ela

- **master** É a branch que contém código em nível de produção, ou seja, o código mais maduro existente na sua aplicação.
- **develop** - É a branch que contém código em nível preparatório para o próximo deploy. Ou seja, quando features são terminadas, elas são juntadas com a branch develop, testadas (em conjunto, no caso de mais de uma feature), e somente depois as atualizações da branch develop passam por mais um processo para então ser juntadas com a branch master;
- **feature** São branches nas quais são desenvolvidos recursos novos para o projeto em questão.
- **test** São branches nas quais realizamos testes, sejam de back, front ou de integração.
- **bugfix** São branches nas quais são realizadas correções de bugs críticos encontrados, ou correções de textos, cores, etc.
- **refactor** São branches nas quais realizamos mudanças internas no nosso código que não modificam a sua forma de funcionamento externa, apenas aprimoram a qualidade do código.

Exemplos: *feature/front/pedro/onBoarding refactor/back/pedro/clientModel update/front/pedro/menu config/back/pedro/env*

Commits

git commit -m "tipo: descrição"

- **init:** Utilizado quando damos o primeiro commit do projeto.
- **config:** Utilizado quando adicionamos/atualizamos as configurações do projeto. Por exemplo, quando editamos o arquivo `.env` para adicionar o nome da base de dados, nome de usuário e senha.
- **docs:** Utilizado quando adicionamos ou atualizamos a documentação do projeto. Por exemplo, quando atualizamos o README.
- **feature:** Utilizado quando adicionamos uma nova funcionalidade no projeto. Por exemplo, no front-end, quando criamos a página de login, e no back-end, quando criamos o fluxo de autenticação de usuários.
- **add:** Utilizado quando instalamos alguma dependência ou arquivo no projeto. Por exemplo, uma dependência de máscaras para formulários.
- **remove:** Utilizado quando houver a remoção de dependência, funcionalidade ou arquivo. Por exemplo, remover uma dependência de formulário.
- **integration:** Utilizado quando realizamos integração de funcionalidades no back e front. Por exemplo, quando integramos a página de login.

- **merge:** Utilizado quando fazemos o merge de duas branches. Por exemplo, quando realizamos o merge de uma branch de front e uma branch de back para realizarmos a integração.
- **bugfix:** Utilizado quando houver a correção de bugs. Por exemplo, botão encaminhando para página errada.
- **refactor:** Utilizado quando realizamos mudanças internas no nosso código que não modificam a sua forma de funcionamento externa, apenas aprimoram a qualidade do código. Por exemplo, quando removemos a repetição de um trecho de código e criamos uma função para realizar o trabalho.
- **test:** Utilizado quando adicionamos ou atualizamos testes unitários no projeto. Por exemplo, quando adicionamos um teste para o fluxo de login de usuários.